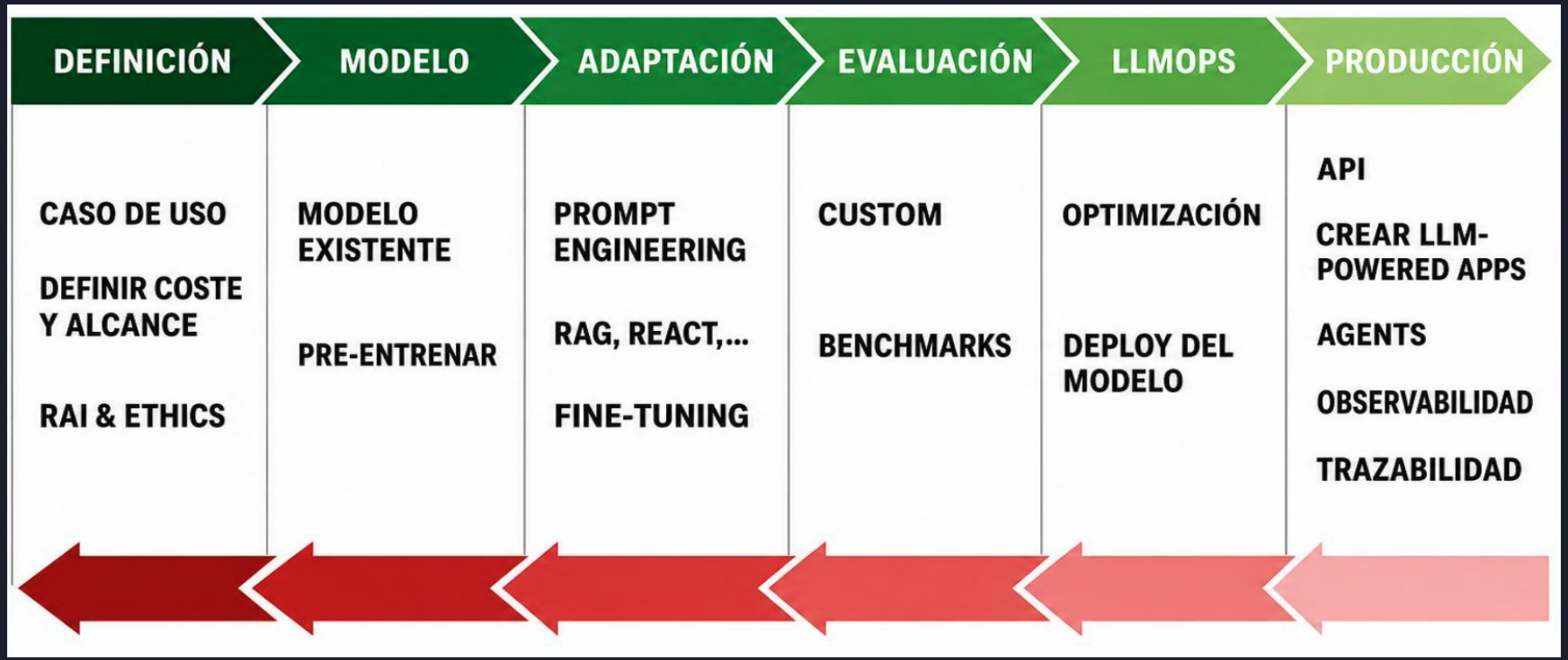
keep coding

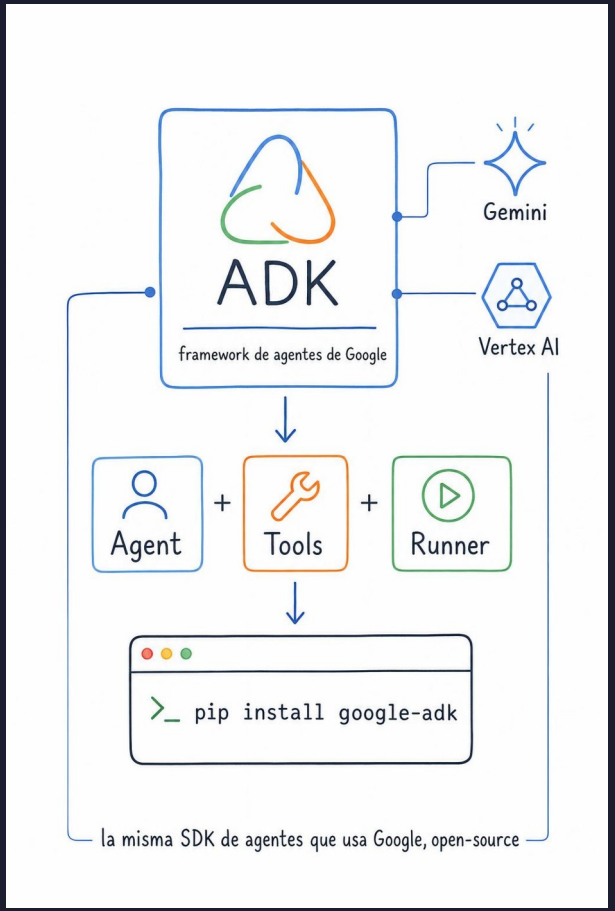
AI ENGINEERING

Agent Development Kit


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

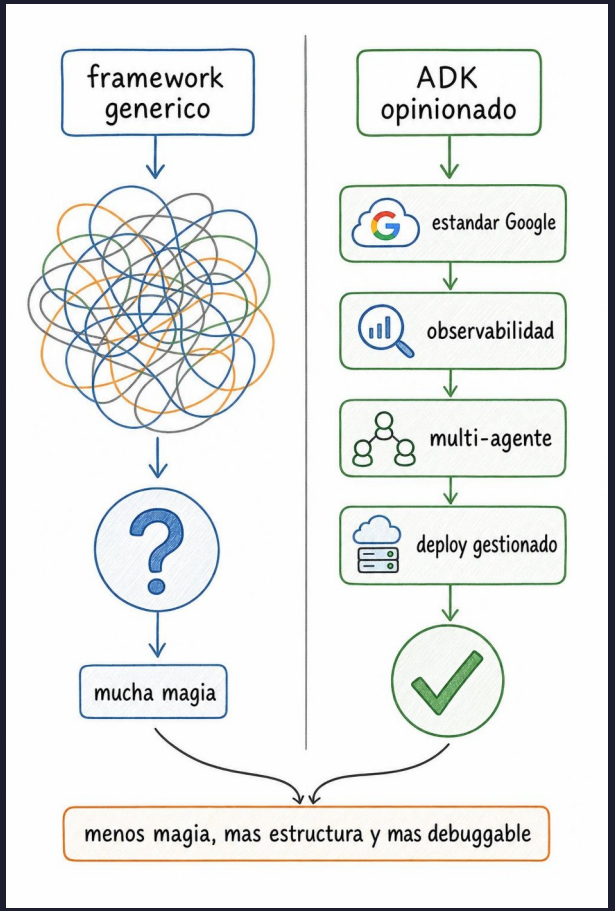
¿Qué es ADK?

- ADK · framework open-source de Google para agentes con LLMs
- `pip install google-adk` · Python · misma SDK que usa Google
- Modelo mental explícito · Agent + Tools + Runner + Session
- Gemini y Vertex nativos · LiteLLM para OpenAI, Claude, Llama
- Production-ready desde el día uno · no una demo de juguete



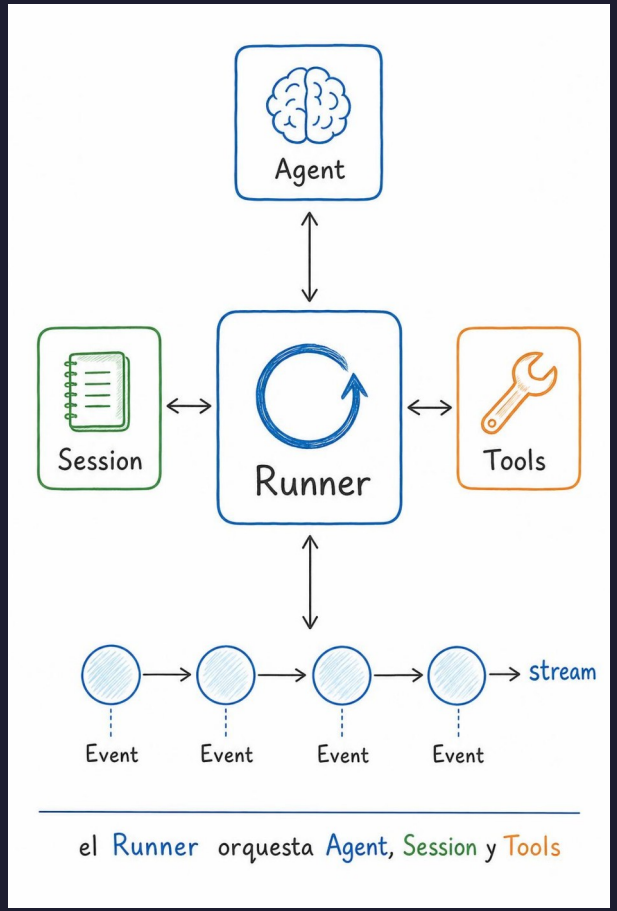
Motivación

- Un framework opinionado · decide la estructura por ti
- Estándar Google · alineado con Gemini, Vertex y A2A protocol
- Observabilidad built-in · cada paso es un Event tipado y trazable
- Multi-agente y deploy gestionado de serie · no como plugin
- Menos magia que Langchain · más debuggable y predecible



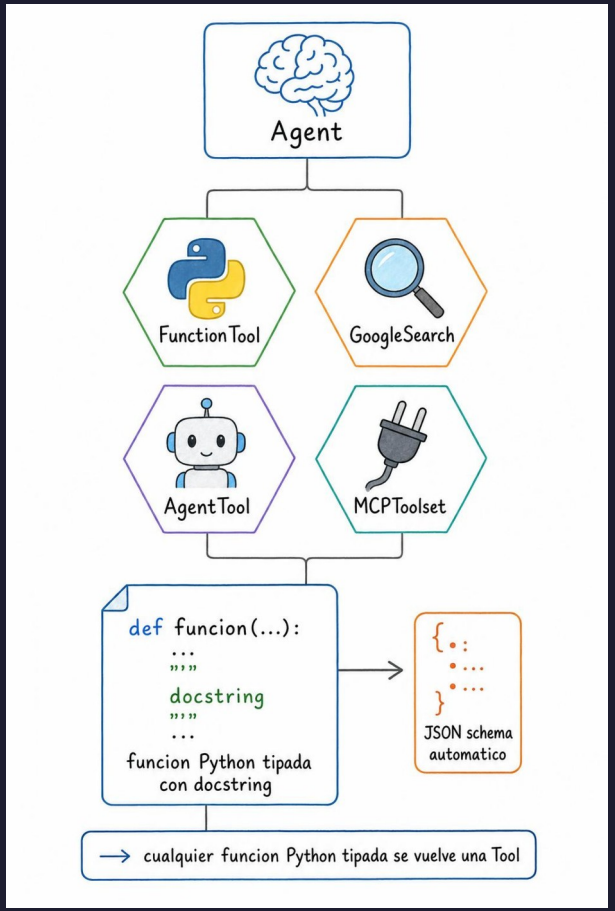
Arquitectura interna

- Hub-and-spoke · el Runner orquesta Agent, Session y Tools
- Agent · LLM + instrucción + tools · decide la siguiente acción
- Session · estado persistente · `state: dict` + `events: list`
- Event · cada paso (input, llm, tool, output) es un Event tipado
- El Runner es un loop async · itera hasta la respuesta final



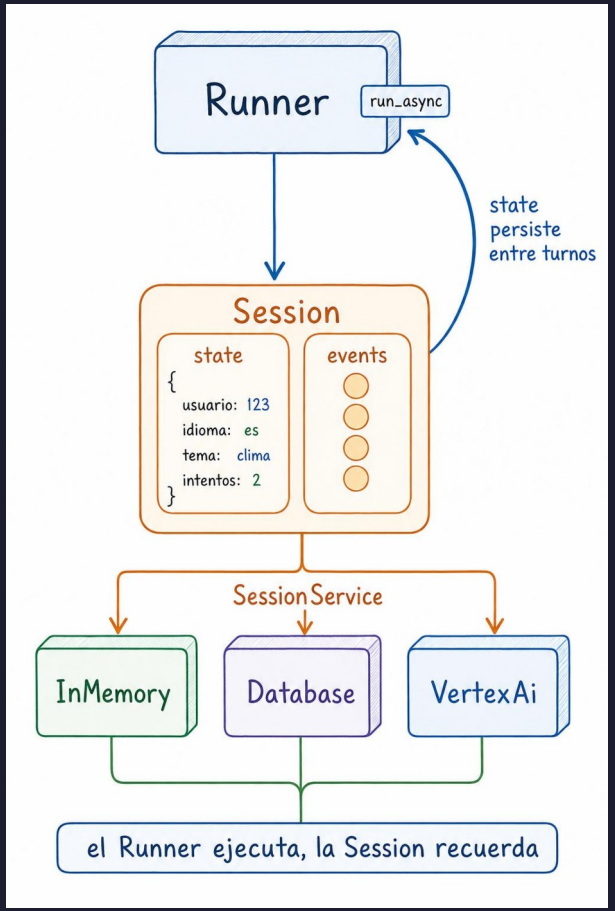
Primitivo · Tool

- `FunctionTool` · cualquier función Python tipada · auto-schema
- El LLM lee el docstring para decidir cuándo llamar la tool
- Built-in · `GoogleSearch`, `CodeExecution`, `VertexAiSearch`
- `AgentTool` · envuelve un agente como tool de otro agente
- `MCPToolset` · consume servidores MCP externos directamente



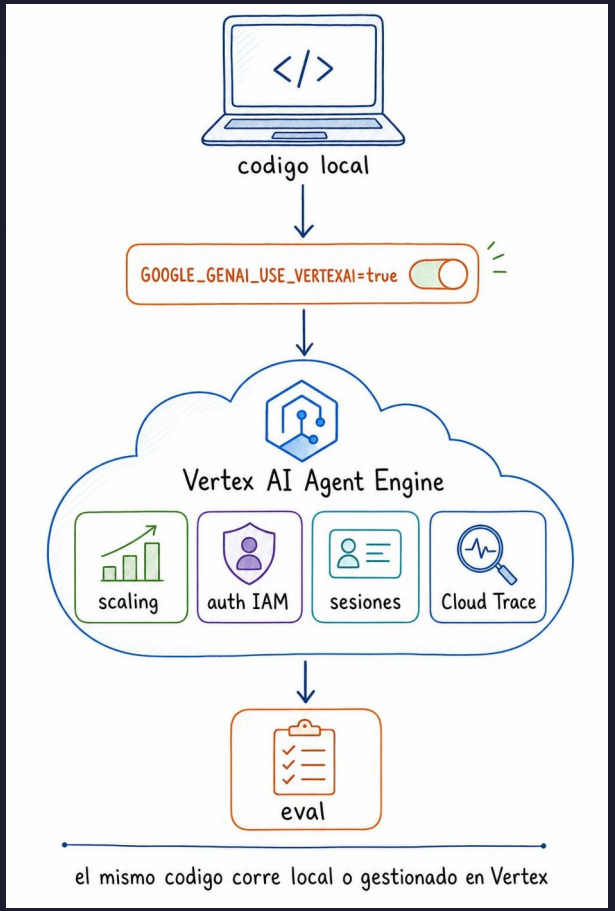
Primitivo · Runner y Session

- `Runner(app_name, agent, session_service)` · orquesta la ejecución
- `run_async(user_id, session_id, new_message)` · yield de events
- Session · `state` (dict) + `events` · vive en un SessionService
- Services · `InMemory`, `Database` (SQLite/Postgres), `VertexAi`
- El state persiste entre turnos · accesible desde las tools



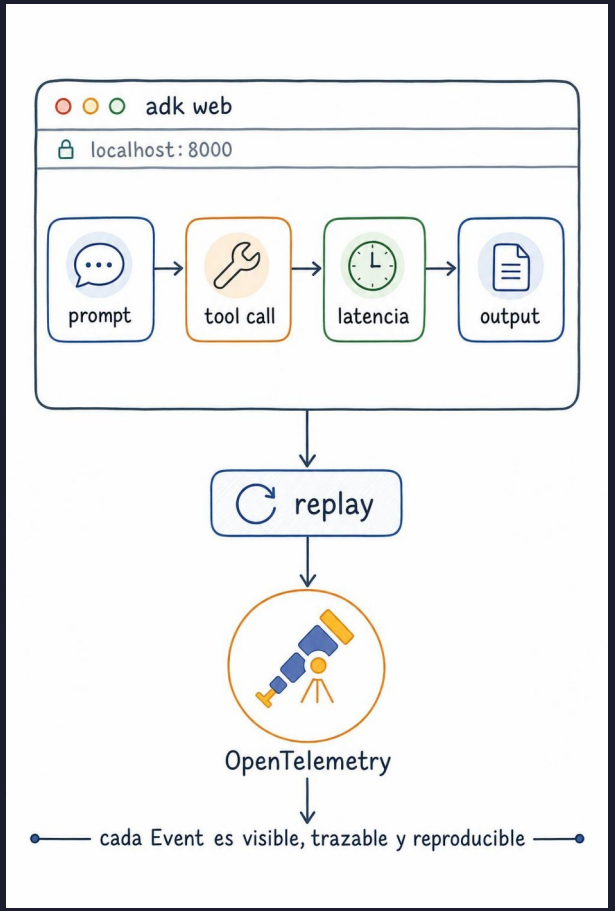
Integración con Vertex AI

- `GOOGLE_GENAI_USE_VERTEXAI=true` · cambia backend sin tocar código
- Vertex AI Agent Engine · runtime gestionado · scaling y auth
- `agent_engines.create(agent, requirements)` · deploy en un comando
- `VertexAiSessionService` · sesiones persistentes y multi-region
- Mismo código local y cloud · solo cambia el SessionService



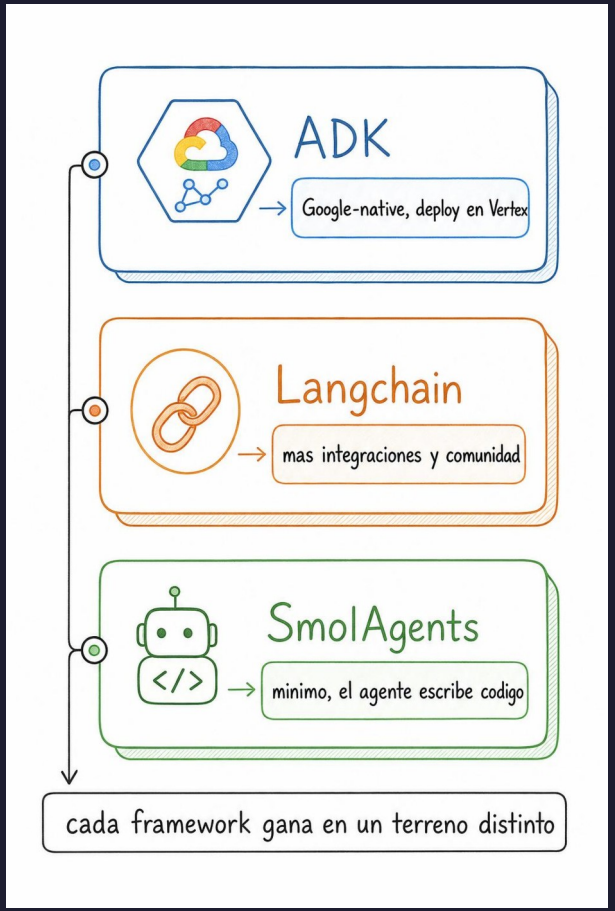
Observabilidad · adk web y eventos

- `adk web` · dashboard en `localhost:8000` para dev local
- Cada Event es un nodo · ves prompt, tool calls y latencias
- Replay · reproduces la sesión completa paso a paso
- Trace export a OpenTelemetry · Langfuse, Arize, Phoenix
- Si no lo puedes inspeccionar, no lo puedes depurar



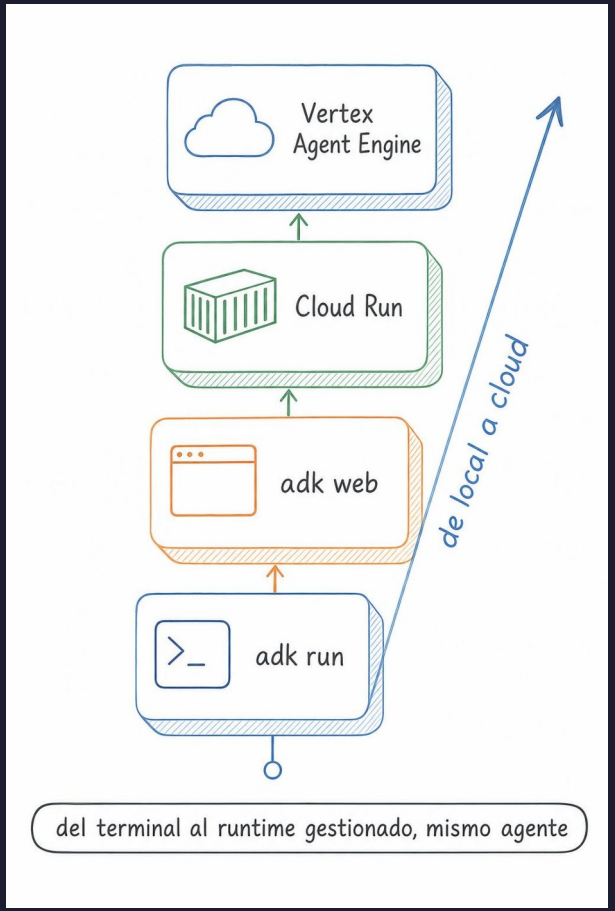
ADK vs Langchain vs SmolAgents

- ADK · opinionado, Google-native, multi-agent y A2A de serie
- Langchain · ecosistema mayor, miles de integraciones, flexible
- SmolAgents · code-first, mínimo, el agente escribe Python
- ADK gana en · deploy en Vertex, observabilidad, sesiones
- Elige por contexto · no hay framework ganador universal



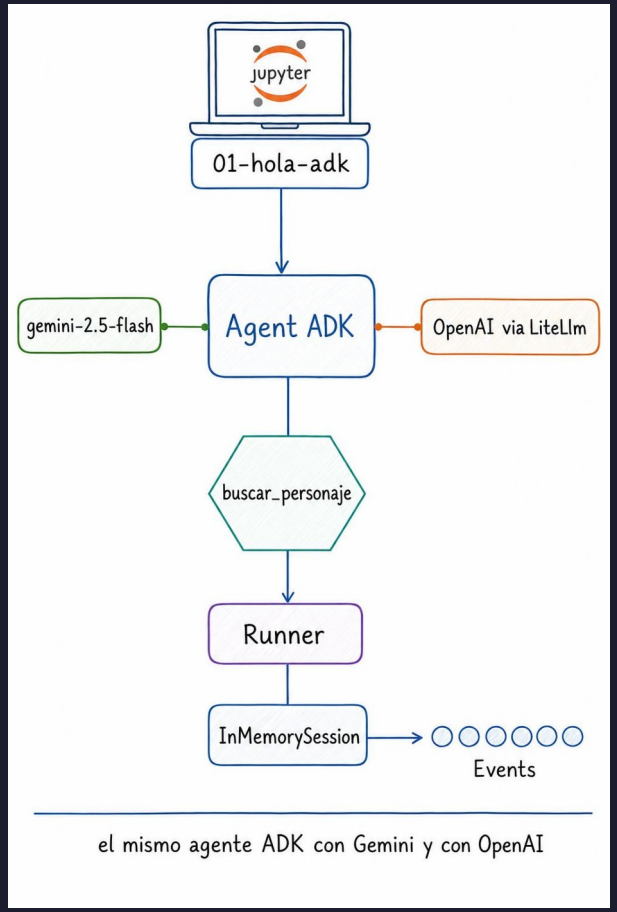
Deploy · local y cloud

- Local · `adk run agent_dir` · CLI interactiva en terminal
- Local web · `adk web` · chat y debugger en localhost:8000
- API server · `adk api_server` · expone una REST API
- Cloud · `adk deploy cloud_run` o `adk deploy agent_engine`
- Del portátil a Vertex Agent Engine sin reescribir el agente



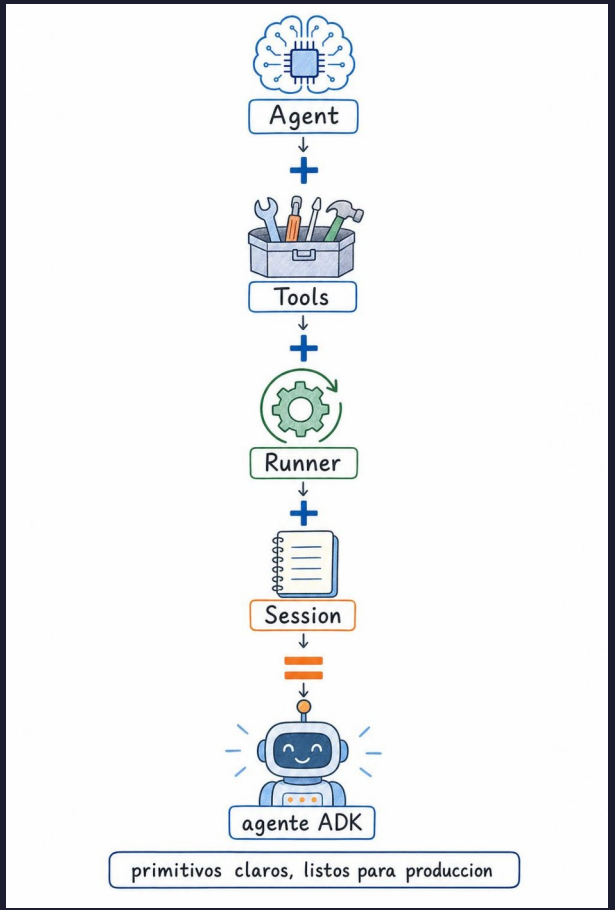
LAB · agente ADK en un notebook

- Todo en el notebook, en local · `pip install google-adk litellm`
- Tool tipada `buscar_personaje_onepiece` · ADK le saca el schema solo
- Mismo Agent con `gemini-2.5-flash` y con OpenAI vía `LiteLlm`
- Runner + `InMemorySessionService` · iteramos Events en vivo
- ADK es agnóstico de modelo · cambias una línea



Recap

- ADK = framework de agentes de Google · production-grade
- Primitivos claros · Agent, Tool, Runner, Session, Event
- Multi-agente nativo · Sequential, Parallel, Loop, AgentTool
- Observabilidad de serie · `adk web`, eventos, trazas OTel
- Best-in-class si vives en GCP · combinable con MCP y A2A




keep coding